Project review — 5 September 2026

The project has **18 actionable findings: 11 P1 issues and 7 P2 issues**. The P1 issues should be resolved before a shared production deployment: uploaded kubeconfigs can invoke commands in the API process, several tenant boundaries are incomplete, and deployed scanner workers cannot decrypt the API's saved credentials. The pinned AI dependencies also prevent startup when AI is configured.

This review covered the repository structure, CloudGuard API and scanner runtime, credentials, jobs and sandbox lifecycle, SQL ownership boundaries, GRC authentication and representative service operations, both frontends, deployment variants, and test/CI configuration. The inventory contains 1,691 tracked files. Findings below come from source inspection and focused local probes; this is a broad review, not a claim that every line or integration has been exhaustively tested. Presentation layouts, generated media, live cloud resources, and deployed production behavior were not validated.

P1 means a high-priority security or operational defect. P2 means a functional, reliability, or development-workflow defect. No application source or existing user artifacts were changed during the review.

1. [P1] Uploaded kubeconfigs can execute commands during credential validation

kubernetes_server.py:76, credentials/api.py:302, credentials/manager.py:790

The upload endpoint accepts a YAML object containing `clusters` and `users`, then schedules validation. `build_kubernetes_api_client()` passes it directly to `KubeConfigLoader.load_and_set()`. A supplied `users[].user.exec` configuration therefore reaches the Kubernetes client's executable authentication provider before any successful cluster connection is required. A local probe with `ExecProvider.run` mocked observed one invocation; the installed provider uses `subprocess.Popen`. No command from the test configuration was executed. This is also documented in the Kubernetes guidance on untrusted kubeconfigs.

Reject executable authentication configurations and server-local file references before loading tenant input. If credential plugins are required, run an explicitly controlled implementation in an isolated worker with only the requesting tenant's credentials.

2. [P1] The pinned OpenAI/HTTPX versions prevent AI-enabled startup

requirements.txt:38, main.py:231

The project pins `openai==1.54.3` with `httpx==0.28.1`. Using those exact locally installed versions, constructing `OpenAI(api_key='review-placeholder-not-a-real-key')` raises `TypeError: Client.__init__() got an unexpected keyword argument 'proxies'`, without making a network request. `main.py` constructs this client at import time when an OpenAI key, AI key, or AI base URL is configured. Consequently the API and workers importing `backend.main` fail before serving requests.

Choose compatible SDK/HTTP-client versions and add an import/startup check that supplies a dummy AI configuration without contacting an external endpoint.

3. [P1] Scanner and sandbox workers lack the API's credential encryption key

Dokploy worker environment:150, E2E worker environment:13, credential decryption:99

The API receives `ENCRYPTION_KEY`, but the Dokploy and E2E local scan/sandbox worker containers do not. DigitalOcean's scheduler and sandbox worker also omit it. Credentials written by the API have the `v1:` encrypted format, and workers cannot read them. A local probe using the actual encryption/decryption methods reproduced `ValueError: ENCRYPTION_KEY is required to decrypt stored credentials`. Some retrieval paths then replace undecryptable fields with `None`, obscuring the configuration failure as missing/invalid cloud credentials.

Pass the same required encryption key to every process that reads or writes cloud credentials and fail startup if it is missing. Verify save-in-API/read-in-worker behavior with an encrypted dummy credential.

4. [P1] Two documented production paths leave browser/API authentication disabled

DigitalOcean backend environment:25, root production backend:222, authentication switch:41

The DigitalOcean and root production Compose services do not pass `CLOUDGUARD_AUTH_REQUIRED` or the Keycloak verification settings. The authentication helper defaults to `false`; setting `APP_ENV=production` does not change it. `WORKER_AUTH_REQUIRED` protects machine routes only. Following these production guides and exposing the API through the suggested reverse proxy therefore leaves application requests on the unauthenticated/session-cookie compatibility path. Dokploy and E2E portal configurations explicitly enable authentication and do not have this particular omission.

Require browser authentication and complete Keycloak configuration in every production service definition, with a production startup assertion that rejects disabled authentication.

5. [P1] Raw MCP endpoints reuse another user's provider session

main.py:7115, main.py:7146, provider registry:356

Cloud provider servers are stored in a process-wide dictionary keyed only by provider. User scan initialization places credential-bearing servers in that dictionary, but `/api/mcp/servers/{provider}/request` and `/tools/{tool_name}` dispatch directly to it without resolving the calling user or checking server ownership. Even with JWT authentication enabled, a different signed-in user can invoke the provider session currently registered in that API process and receive its results. Shared registration also creates cross-request credential races.

Bind provider instances to verified tenant/user/job ownership, and resolve that ownership on every tool/resource request. Restrict or remove raw provider-management endpoints from the general user API.

6. [P1] Evidence ID collisions overwrite a different tenant's artifact

evidence upsert:4218, ingestion endpoint:4785, evidence uniqueness:32

Connector identity is verified, but `evidence_id` is client-supplied and globally unique. `ON CONFLICT (evidence_id) DO UPDATE` replaces the artifact payload, URI, checksum, metadata, and job association without comparing tenant/user ownership. An authenticated connector that reuses a known ID can corrupt another tenant's evidence; accidental cross-tenant ID reuse has the same effect. An isolated SQLite probe using the exact upsert statement, with parameter placeholders adapted, retained `tenant-a/user-a` ownership while replacing the payload and source with tenant B's values.

Make the conflict key tenant-scoped, or enforce matching ownership in the upsert and reject foreign conflicts. Validate referenced jobs against the same connector tenant before linking them.

7. [P1] Persistent findings and remediation tasks are globally readable and writable

memory endpoints:6508, task update:6584, memory schema:101

The memory endpoints authenticate the request through middleware but never use its tenant or user. Dashboard/task reads query global memory, and task status changes use only the integer task ID. These memory tables have no tenant ownership columns; finding hashes also omit tenant/account ownership. Once enhanced scans populate them, any authenticated user can enumerate other users' remediation tasks and update their status. Unrelated findings can also be conflated across tenants.

Persist tenant ownership throughout memory, task, and note records; include it in finding identity and every read/update. Add cross-tenant read and write regression cases.

8. [P1] GRC runbook child operations bypass parent tenant checks

runbook step endpoints:122, runbook step deletion:389, plan-control deletion:435

Top-level runbook retrieval receives `organizationId`, but step mutation routes do not. The update/delete SQL filters only by runbook ID and step number, with no parent ownership lookup. The controller applies JWT authentication without a role restriction, so a signed-in viewer knowing another organization's runbook ID can change or delete its recovery steps. A probe of the actual `deleteStep` method produced both deletion and renumbering queries without any tenant condition. The plan-control unlink path has the same missing parent ownership boundary.

Resolve and authorize the parent in the caller's organization before every child operation, require the appropriate write permission, and include parent ownership in mutation queries. Audit the analogous business-process and communication-plan child routes as part of that fix.

9. [P1] GRC requirement mutations can modify another tenant's framework

requirement endpoints:203, requirement creation:193, requirement update:557

Requirement creation, bulk upload, and update check role but omit the caller's organization.

Creation calls `findOne(frameworkId)` without its optional organization argument; updates check only framework and requirement IDs. An administrator/compliance manager from one tenant can therefore alter a known private framework belonging to another. A probe confirmed the update lookup filters only on `{id, frameworkId}` and the write only on `{id}`.

Require organization context in these service signatures and authorize ownership of the framework. Keep global framework catalog mutation under a separate explicit policy.

10. [P1] Read-only users can enable and administer external audit access

portal user creation:93, portal provisioning:430

The portal administration endpoints use only `JwtAuthGuard`. There are no role/permission requirements on create, update, delete, or bulk provisioning. The service checks organization membership but not the caller's role. A viewer can create an external portal account, choose its portal permissions, receive its plaintext access code, and automatically enable the audit portal. This grants durable external access despite the viewer's read-only role.

Require audit-access administration permission on every portal management operation, including bulk creation. Test that viewers and non-administrative auditors receive 403 responses.

11. [P1] Cleanup exceptions strand vulnerable labs in `destroying`

cleanup state transition:3577, cleanup call:3589, reconciliation selection:3651

Cleanup changes the persistent state to `destroying` before calling the provider or storing cleanup evidence. An exception escapes without resetting the state. Reconciliation excludes `destroying`, and the completion query only selects `scanning`, so future worker iterations do not retry the lab. A local probe forced a provider cleanup exception and observed the final state remain `destroying`. A worker crash during cleanup has the same recovery gap. Temporary vulnerable resources may therefore outlive the advertised TTL.

Persist a retriable cleanup failure state and reclaim stale `destroying` work with a lease. Keep cleanup idempotent and make provider failures recoverable even when evidence storage also fails.

12. [P2] Provider failures are marked completed and bypass job retry

provider exception handling:1043, job completion:4758, stored scan completion:7233

The scan runner catches provider exceptions and returns an error-bearing `ScanResult`. The job processor then always sets `completed` and clears `error`/`last_error`; the scan record is likewise finished without using its errors. With the actual runner/job methods and a provider stub raising `review-provider-offline`, the result was `status=completed, error=null`, while the nested provider errors still contained the failure. The queue's retry/dead-letter branch is never reached for that failure.

Distinguish successful, partial, and failed scans. Preserve provider errors at job level and route retriable failures through the attempt policy.

13. [P2] Selected GCP/Kubernetes credentials are ignored during scans

GCP initialization:754, Kubernetes initialization:794, sandbox scan request:3487

`initialize_mcp_servers_for_user()` honors `credential_id` for AWS but always chooses the default credential for GCP and Kubernetes. Sandbox provisioning accepts and uses the selected credential, then queues a scan carrying that same ID, which initialization discards. With a nondefault sandbox credential, the scan can inspect the default project/cluster instead of the lab just created, and cleanup can proceed without validating the intended environment.

Resolve the requested credential for each supported provider, verify its ownership and provider type, and use a default only when no ID was supplied.

14. [P2] Authentication blocks normal navigation to history and schedules

middleware path classification:159, history link:1241, schedules navigation:807

The middleware classifies `/history`, `/scheduled_scans`, and `/schedules` as bearer-protected APIs, but they serve HTML pages. The frontend stores its token in session storage and adds it through a fetch wrapper. Ordinary links and `window.location.href` do not attach that bearer token, so the browser receives 401 before page JavaScript can initialize authentication. A local middleware probe returned 401 for all three paths and 200 for `/dashboard`.

Serve the page shell through the same authentication bootstrap as other HTML pages, keeping its data APIs protected, or implement authenticated server sessions that work for navigation.

15. [P2] Custom fields, retention policies, and delegations disappear on restart

custom field stores:53, retention policy store:38, delegation store:33

These production modules acknowledge creation and updates but store the records only in module-level Maps. They have no persistent write/read path for those records. Restarting or deploying the controls service loses field definitions and values, retention configuration, and delegation records; multiple replicas also see inconsistent state. These modules are registered in the application rather than restricted to a demo mode.

Persist these records in tenant-scoped database tables and test retrieval after recreating the service process.

16. [P2] The merged repository's GitHub workflows are inactive

PR workflow:1, image build workflow:123

All four tracked workflows are under `grc-platform/.github/workflows`; there is no repository-root `.github/workflows`. GitHub discovers workflows in the root location described in its workflow syntax documentation. Thus these files do not provide PR validation, image builds, deployment, or security checks for this merged repository. Simply moving them is insufficient: their package-lock paths, working directories, and build contexts still assume GRC is the repository root.

Move/adapt the workflows to the merged root, correct their paths, and add CloudGuard's Python tests to the actual validation workflow.

17. [P2] The frontend's required TypeScript build check fails

CollectorConfigModal.tsx:184

The create/update mutation returns a union of Axios response types parameterized by different request DTOs. React Query infers the create response type, so the update branch violates the mutation function signature. The frontend's own TypeScript 5.9.3 reports TS2322 here, blocking `build:typecheck` and the configured frontend compilation check. All seven backend/shared TypeScript projects passed using their own installed compiler versions.

Normalize the mutation result, for example by returning the response body with an explicit shared result type, and keep the TypeScript check in the release path.

18. [P2] The documented quick start does not start the application

README quick start:96, development service profile:157, merge guide:99

Both guides instruct users to run `docker compose up --build`, but application services are behind Compose profiles. With no profile enabled, `docker compose --env-file /dev/null config --services` returns only `postgres` and `redis`. A fresh user following the instructions gets no API/dashboard. The README also alternates between port 8000 and the default HTTP port for the dashboard URL.

Document and validate the intended development profile command and its actual URL, or make the expected application service part of the default Compose configuration.

Validation performed

Check	Result
CloudGuard Python tests	10 passed
GRC frontend unit tests	34 files, 397 tests passed
GRC shared tests	61 passed, 6 failed
GRC controls tests	316 passed, 6 failed
GRC frameworks tests	133 passed, 15 failed
GRC policies tests	21 passed
GRC TPRM tests	12 passed
GRC trust tests	11 passed
GRC audit tests	26 passed
Backend/shared TypeScript checks	All seven projects passed with their workspace compilers
Frontend TypeScript check	Failed: one TS2322 diagnostic, finding 17
Python syntax	49 tracked Python files parsed successfully
Shell syntax	31 tracked shell files passed `bash -n`

Compose structure	Nine root/deployment/GRC configurations passed validation without environment interpolation

The GRC backend/shared total is **580 passed and 27 failed**. Failures are concentrated in test maintenance: obsolete proxy/header-auth expectations, permission names that no longer match the implementation, an encoded-path expectation, a missing `all-good.xlsx` import fixture, and missing `STORAGE_PROVIDER` mocks. Those failures should be repaired without restoring insecure header-based authentication. A failing encoded-path expectation alone does not demonstrate an exploitable filesystem traversal.

Python tests ran with `/Users/anjali/anaconda3/bin/python3.11`; the default `/usr/local/bin/python3` lacks pytest and several backend dependencies. Initial checks using the root TypeScript 6 installation hit configuration deprecations; those were superseded by checks using each workspace's installed TypeScript 5.9.3. Backend test suites also load the existing shared build through workspace package resolution, so a clean rebuilt test run remains worthwhile after fixes.

Focused probes confirmed kubeconfig execution-provider invocation, AI client construction failure, missing-key decryption failure, evidence conflict behavior, HTML navigation rejection, failed-provider job completion, stranded cleanup state, and absent GRC tenant filters. They used dummy values and mocks/extracted source methods rather than live application data. The evidence SQL probe used SQLite rather than PostgreSQL and preserved the upsert's conflict/ownership behavior.

A complete Docker build, live Keycloak/browser session, real cloud scan, and PostgreSQL migration run were not completed. Docker's daemon was unavailable. An isolated PostgreSQL initialization attempt failed because the sandbox disallowed the required shared-memory operation; no production database was contacted. Compose validation checks structure, not the correctness or availability of deployed credentials/services.

Detailed test and type-check logs are in the local review log directory. The highest-value follow-up checks are multi-tenant HTTP tests for the affected routes, a startup test with AI configured, an encrypted credential handoff between containers, and crash/retry tests for the scan and sandbox workflows.